Modül 10

HttpClient

API Communication & HTTP

Modül:	10 / 30
Ders Sayısı:	8 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

🚀 Modül 10'a Hoş Geldin

Backend ile konuşmadan modern web uygulaması olmaz. Bu modülde Angular'ın HttpClient'ını, modern functional interceptor'ları, error handling pattern'lerini ve file upload/download gibi advanced senaryoları öğreneceğiz.

Bu modülün sonunda yapabileceklerin:

- ✓ HttpClient kurulumu ve modern configuration
- ✓ GET/POST/PUT/DELETE istekleri yapmak
- ✓ Functional interceptor'lar yazmak (auth, logging, error)
- ✓ HttpResource ile signal-based API
- ✓ Error handling stratejileri
- ✓ File upload/download (progress events ile)
- ✓ HttpContext ile metadata geçirmek
- ✓ Production-ready service patterns

DERS 10.1

HttpClient Kurulumu

Modern Angular'da HttpClient nasıl configure edilir.

⚙ Standalone Setup

```
// app.config.ts
import { provideHttpClient, withFetch, withInterceptors } from
'@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(),  // Fetch API kullan (XMLHttpRequest yerine)
      withInterceptors([authInterceptor, errorInterceptor]),
    )
  ]
};
```

withFetch() — Modern

- ✓ Browser'ın native Fetch API'sini kullanır
- ✓ AbortController desteği (auto cancellation)
- ✓ SSR uyumlu
- ✓ Daha küçük bundle

Use in Components/Services

```
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);

  getUsers() {
    return this.http.get<User[]>('/api/users');
  }
}
```

DERS 10.2

HTTP Methods

GET, POST, PUT, PATCH, DELETE — temel HTTP işlemleri.

GET

```
// Basit
this.http.get<User[]>('/api/users');

// Query params
this.http.get<User[]>('/api/users', {
  params: { page: '1', limit: '10', sort: 'name' }
});

// HttpParams (programatik)
const params = new HttpParams()
  .set('page', 1)
  .set('limit', 10)
  .append('tag', 'admin')   // birden fazla aynı key
  .append('tag', 'active');
this.http.get<User[]>('/api/users', { params });

// Headers
this.http.get<User[]>('/api/users', {
  headers: { 'X-Custom-Header': 'value' }
});
```

POST/PUT/PATCH

```
// POST — yeni oluşturma
this.http.post<User>('/api/users', {
  name: 'John',
  email: 'j@example.com'
});

// PUT — tüm field'ları güncelleme
this.http.put<User>(`/api/users/${id}`, fullUserObject);

// PATCH — kısmi güncelleme
this.http.patch<User>(`/api/users/${id}`, {
  email: 'new@example.com'
});

// Headers ile
this.http.post<User>('/api/users', body, {
  headers: { 'Content-Type': 'application/json' }
});
```

DELETE

```
this.http.delete(`/api/users/${id}`);
this.http.delete<{ success: boolean }>(`/api/users/${id}`);
```

Response Types

```
// Default — JSON parse
this.http.get<User>('/api/user');  // User object

// Text
this.http.get('/api/text', { responseType: 'text' });  // string

// Blob (file)
this.http.get('/api/file', { responseType: 'blob' });  // Blob

// Full response (headers, status dahil)
this.http.get<User>('/api/user', { observe: 'response' });
// → HttpResponse<User>
```

DERS 10.3

Functional Interceptors

Modern Angular'da class-based interceptor deprecated. Functional kullanıyoruz.

Auth Interceptor

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getToken();

  if (token) {
    req = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
  }

  return next(req);
};
```

Logging Interceptor

```
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const start = Date.now();
  console.log(`→ ${req.method} ${req.url}`);

  return next(req).pipe(
    tap(event => {
      if (event.type === HttpEventType.Response) {
        const duration = Date.now() - start;
        console.log(`← ${req.method} ${req.url} [${event.status}] ${duration}ms`);
      }
    })
  );
};
```

⚠ Error Interceptor

```
import { catchError, throwError } from 'rxjs';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const toast = inject(ToastService);

  return next(req).pipe(
    catchError(error => {
      if (error.status === 401) {
        router.navigate(['/login']);
      } else if (error.status === 403) {
        toast.error('Yetkiniz yok');
      } else if (error.status >= 500) {
        toast.error('Sunucu hatası');
      }
      return throwError(() => error);
    })
  );
};
```

Retry Interceptor

```
import { retry, timer } from 'rxjs';

export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  // Sadece GET için retry
  if (req.method !== 'GET') return next(req);

  return next(req).pipe(
    retry({
      count: 3,
      delay: (error, retryCount) => {
        // 5xx hatalarda retry, exponential backoff
        if (error.status >= 500) {
          return timer(1000 * Math.pow(2, retryCount));
        }
        throw error;
      }
    })
  );
};
```

Sıra Önemli

```
provideHttpClient(
  withInterceptors([
    loggingInterceptor,    // 1. Log
    authInterceptor,       // 2. Auth header ekle
    retryInterceptor,      // 3. Retry logic
    errorInterceptor,      // 4. Error handling
  ])
)
```

DERS 10.4

httpResource — Modern Async

Modül 5'te öğrendiğimiz httpResource'u API context'inde derinlemesine işliyoruz.

□ Klasik vs Modern

```
// □ Klasik subscribe pattern
@Component({...})
export class UserComponent implements OnInit {
  user = signal<User | null>(null);
  loading = signal(false);
  error = signal<string | null>(null);

  ngOnInit() {
    this.loading.set(true);
    this.http.get<User>('/api/users/123').subscribe({
      next: u => { this.user.set(u); this.loading.set(false); },
      error: e => { this.error.set(e.message); this.loading.set(false); }
    });
  }
}

// ✓ Modern httpResource
@Component({...})
export class UserComponent {
  userId = signal('123');

  user = httpResource<User>(() => `/api/users/${this.userId()}`);
  // user.value()    — User | undefined
  // user.isLoading() — boolean
  // user.error()    — unknown
  // user.status()   — 'idle' | 'loading' | 'resolved' | 'error'
}
```

□ Template Pattern

```
@switch (user.status()) {
  @case ('loading') { <app-spinner /> }
  @case ('error') {
    <p>Hata: {{ user.error() }}</p>
    <button (click)="user.reload()">Tekrar Dene</button>
  }
  @case ('resolved') {
    @let u = user.value()!;
    <h1>{{ u.name }}</h1>
  }
}
```

DERS 10.5

Error Handling Patterns

Hataları sistematik şekilde yönetmek.

Global Error Strategy

```
// 1. Error Interceptor (network level)
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const toast = inject(ToastService);

  return next(req).pipe(
    catchError(err => {
      // Global error handling
      if (err.status === 0) {
        toast.error('Bağlantı yok');
      } else if (err.status === 401) {
        // Redirect to login
      }

      return throwError(() => err);
    })
  );
};

// 2. Service-level handling
@Injectable()
export class UserService {
  getUsers() {
    return this.http.get<User[]>('/api/users').pipe(
      catchError(err => {
        // Domain-specific handling
        if (err.status === 404) return of([]);
        return throwError(() => err);
      })
    );
  }
}

// 3. Component-level (UI feedback)
@Component({...})
export class UsersComponent {
  users = httpResource<User[]>(() => '/api/users');

  // Template: @if (users.error()) { ... }
}
```

Type-Safe Error

```
import { HttpErrorResponse } from '@angular/common/http';

interface ApiError {
  code: string;
  message: string;
  details?: Record<string, any>;
}

function handleError(err: HttpErrorResponse): never {
  if (err.error instanceof ProgressEvent) {
    // Network error
    throw { code: 'NETWORK_ERROR', message: 'Bağlantı yok' };
  }

  const apiError = err.error as ApiError;
  throw apiError;
}

// Service
getUsers() {
  return this.http.get<User[]>('/api/users').pipe(
    catchError(handleError)
  );
}
```

DERS 10.6

File Upload & Download

Progress event'leri ile dosya yönetimi.

📤 File Upload — Progress İle

```
import { HttpClient, HttpEventType } from '@angular/common/http';

@Injectable({...})
export class FileService {
  private http = inject(HttpClient);

  uploadFile(file: File) {
    const formData = new FormData();
    formData.append('file', file);

    return this.http.post('/api/upload', formData, {
      reportProgress: true,
      observe: 'events'
    }).pipe(
      map(event => {
        if (event.type === HttpEventType.UploadProgress) {
          const progress = Math.round(100 * event.loaded / (event.total || 1));
          return { type: 'progress', progress };
        }
        if (event.type === HttpEventType.Response) {
          return { type: 'response', body: event.body };
        }
        return null;
      })
    );
  }
}
```

🧩 Component Kullanımı

```
@Component({...})
export class UploadComponent {
  private fileService = inject(FileService);
  progress = signal(0);
  uploading = signal(false);

  onFileSelect(event: Event) {
    const file = (event.target as HTMLInputElement).files?.[0];
    if (!file) return;

    this.uploading.set(true);
    this.fileService.uploadFile(file).pipe(
      takeUntilDestroyed()
    ).subscribe(event => {
      if (event?.type === 'progress') {
        this.progress.set(event.progress);
      } else if (event?.type === 'response') {
        this.uploading.set(false);
        console.log('Upload done:', event.body);
      }
    });
  }
}
```

File Download

```
downloadFile(url: string, filename: string) {
  this.http.get(url, { responseType: 'blob' }).subscribe(blob => {
    const objectUrl = window.URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = objectUrl;
    a.download = filename;
    a.click();
    window.URL.revokeObjectURL(objectUrl);
  });
}
```

DERS 10.7

HttpContext — Metadata Geçirme

Interceptor'lara request bazında metadata göndermek.

Token Tanımla

```
import { HttpContextToken } from '@angular/common/http';

// Skip auth (login isteği auth header istemez)
export const SKIP_AUTH = new HttpContextToken<boolean>(() => false);

// Cache duration
export const CACHE_FOR = new HttpContextToken<number>(() => 0);

// Show loading spinner?
export const SHOW_SPINNER = new HttpContextToken<boolean>(() => true);
```

Request'te Set Et

```
import { HttpContext } from '@angular/common/http';

// Login isteği auth header eklemesin
this.http.post('/api/login', credentials, {
  context: new HttpContext().set(SKIP_AUTH, true)
});

// Public endpoint, spinner gösterme
this.http.get('/api/public/banner', {
  context: new HttpContext()
    .set(SKIP_AUTH, true)
    .set(SHOW_SPINNER, false)
});
```

Interceptor'da Oku

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  // Eğer SKIP_AUTH true ise, header ekleme
  if (req.context.get(SKIP_AUTH)) {
    return next(req);
  }

  const token = inject(AuthService).getToken();
  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }

  return next(req);
};

export const spinnerInterceptor: HttpInterceptorFn = (req, next) => {
  if (!req.context.get(SHOW_SPINNER)) {
    return next(req);
  }

  const spinner = inject(SpinnerService);
  spinner.show();

  return next(req).pipe(finalize(() => spinner.hide()));
};
```

DERS 10.8

Production Patterns

BaseApiService, Cached, Optimistic Update gibi profesyonel pattern'ler.

BaseApiService Pattern

```
@Injectable()
export abstract class BaseApiService<T> {
  protected http = inject(HttpClient);
  protected abstract baseUrl: string;

  getAll(): Observable<T[]> {
    return this.http.get<T[]>(this.baseUrl);
  }

  getById(id: string): Observable<T> {
    return this.http.get<T>(`${this.baseUrl}/${id}`);
  }

  create(data: Partial<T>): Observable<T> {
    return this.http.post<T>(this.baseUrl, data);
  }

  update(id: string, data: Partial<T>): Observable<T> {
    return this.http.patch<T>(`${this.baseUrl}/${id}`, data);
  }

  delete(id: string): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}`);
  }
}

// Kullanım
@Injectable({ providedIn: 'root' })
export class UserService extends BaseApiService<User> {
  protected baseUrl = '/api/users';

  // Özel metodlar
  searchByName(name: string) {
    return this.http.get<User[]>(`${this.baseUrl}/search`, { params: { name } });
  }
}
```

Cached Service Pattern

```
@Injectable({ providedIn: 'root' })
export class CachedUserService {
  private http = inject(HttpClient);
  private cache = new Map<string, User>();

  getUser(id: string): Observable<User> {
    if (this.cache.has(id)) {
      return of(this.cache.get(id)!);
    }

    return this.http.get<User>(`/api/users/${id}`).pipe(
      tap(user => this.cache.set(id, user))
    );
  }

  invalidate(id: string) {
    this.cache.delete(id);
  }

  invalidateAll() {
    this.cache.clear();
  }
}
```

Optimistic Update Pattern

```
@Injectable({ providedIn: 'root' })
export class TodoService {
  private http = inject(HttpClient);
  private _todos = signal<Todo[]>([]);
  todos = this._todos.asReadonly();

  toggleTodo(id: string) {
    // 1. UI'ı hemen güncelle (optimistic)
    const original = this._todos();
    this._todos.update(todos =>
      todos.map(t => t.id === id ? { ...t, completed: !t.completed } : t)
    );

    // 2. Server'a gönder
    this.http.patch(`/api/todos/${id}/toggle`, {}).subscribe({
      error: () => {
        // 3. Hata olursa rollback
        this._todos.set(original);
        alert('Güncelleme başarısız');
      }
    });
  }
}
```

Modül 10 Özeti

HTTP communication artık tam donanımındasın. Modern interceptor'lar, error handling, file upload/download, production pattern'ler — hepsi cebinde.

Neler Öğrendin?

- ✓ provideHttpClient + withFetch + withInterceptors modern setup
- ✓ GET/POST/PUT/PATCH/DELETE — params, headers, response types
- ✓ Functional interceptors: auth, logging, error, retry
- ✓ httpResource() ile signal-based async
- ✓ Error handling — global, service, component katmanlar
- ✓ File upload progress + download
- ✓ HttpContext ile metadata geçirme
- ✓ BaseApiService, Cached, Optimistic Update pattern'leri

Sıradaki Modül

Modül 11 — Resource API Derinlemesine. resource() ve httpResource() ile pagination, search, mutations, error recovery, loading skeletons.